TD n°2 – Les formulaires HTML 5

1 – Introduction

HTML5 a apporté de nombreuses fonctionnalités sur les champs de formulaires. Directement prises en charge par le navigateur, ces fonctionnalités permettent de mettre en œuvre des actions qui nécessitaient auparavant de développer du code JavaScript.

2 – Création d'un formulaire

Un formulaire HTML est réalisé par l'ajout des balises `<form>… </form>`. C'est entre ces 2 balises que seront placés les différents composants du formulaire (*étiquettes de champ, champs de saisie, etc…).*
Le but d'un formulaire est de collecter des informations auprès des utilisateurs pour les envoyer ensuite vers une destination afin de les traiter. Pour ce faire, la balise **`<form>`** dispose de 2 attributs :

- L'attribut **method** : il indique par quel moyen les données vont être envoyées. Deux méthodes peuvent être utilisées au choix :
  - **`method="get"`** : Cette méthode affiche les données transférées dans l'adresse URL et est limitée à 255 caractères.
  - **`method= "post"`** : C'est la méthode la plus utilisée car elle n'affiche pas les données en cours de transfert et n'est pas limitée en nombre d'informations.
- L'attribut **action** : il indique l'adresse de la page qui va traiter les informations. Cette page permettra par exemple d'ajouter les données renseignées dans une base de données. Elle sera écrite dans un langage de programmation comme PHP par exemple.

Le code sera donc le suivant :

```
<body>
        <form method="post" action="traitement.php">
                <p>Composant1 du formulaire</p>
                <p>Composant2 du formulaire</p>
                <p>…</p>
        </form>
</body>
```

Remarque : Le programme "`traitement.php`" sera réalisé ultérieurement.

3 – Les zones de saisie (balise <input>)

3.1 – Saisir dans une zone de texte : `type="text"`

Nous allons débuter notre formulaire par le champ `Nom`. Nous souhaitons que ce champ soit *obligatoire*, c'est-à-dire que l'internaute ne puisse pas valider la saisie du formulaire et en transmettre les informations, tant qu'il n'a pas saisi son nom.
Nous allons donc ajouter le code suivant dans l'élément form :

```
<form method="post" action="traitement.php">
        <p>
                <label for="zoneNom">Nom : </label>
                <input type="text" id="zoneNom" name="nom" required placeholder="Entrez votre
nom" />
        </p>
</form>
```

- La balise `<label>` contient le libellé du champ. Son attribut **`for`** permet de faire le lien avec l'attribut **`id`** de la balise `<input>` et permettra à l'utilisateur de se placer directement dans le champ en cliquant sur le libellé.

- La balise <**input**> décrit le champ de saisie ; ses attributs permettent d'en spécifier les caractéristiques :
  - L'attribut **id** permet d'identifier le champ de saisie. Il est utilisé pour relier le champ de saisie à son libellé et pour les traitements en JavaScript.
  - L'attribut **name** contient la valeur saisie par l'utilisateur dans le champ.
  - L'attribut **required** (HTML5) indique que le champ est obligatoire.
  - L'attribut **placeholder** (HTML5) définit le texte affiché tant que le champ est vierge.

➢ Créer un nouveau dossier **td2** dans votre répertoire personnel puis créer nouvelle page web nommée **index.html.** Saisir dans cette page, le code vu précédemment et ajouter avant la balise </**form**> la ligne suivante :

```
<p><input type="submit" name="envoi" id="envoi" value="Envoyer"/></p>
```

Cette ligne permet de créer le bouton « **Envoyer** ». L'appui sur ce bouton aura pour effet de lancer la vérification de la saisie puis, si tous les champs sont valides, d'effectuer l'action prévue dans **l'attribut action** du formulaire.

➢ Remplacer dans la balise <form>, l'attribut **action="traitement.php"** par **action="envoyer.html".** Pour les besoins du TD, la page « envoyer.html » sera simplifiée.

➢ Créer une nouvelle page **envoyer.html** ne contenant que la ligne suivante« **<h1>Données transférées</h1>** ».

➢ Exécuter la page **index.html**

➢ Cliquer sur Envoyer sans saisir de nom, que constatez-vous ?

……..…….

➢ Idem sous Google Chrome ?

……..

➢ Saisir un nom et cliquer sur Envoyer, que constatez-vous ?

……..……..

➢ Modifier la page **index.html** en remplaçant **method="post"** par **method="get"** et sauvegarder.

➢ Exécuter la page **index.html**, saisir un nom et cliquer sur « Envoyer ».

➢ Noter l'adresse URL affichée :

……..…….

3.2 – Saisir une adresse de courriel : type="email"

Un autre apport important de HTML5 sur les formulaires est la possibilité de spécifier le type de chaque champ et donc de déléguer au navigateur les tâches d'aide à la saisie et de vérification. Ce n'est plus au développeur de créer le code pour réaliser ces opérations.
Le code d'un champ de saisie d'une adresse de courriel en HTML5 sera donc le suivant :

```
<p>
	<label for="zoneMail">E-mail : </label>
	<input type="email" id="zoneMail" name="mail" />
</p>
```

Remarque : En ajoutant l'attribut **`multiple`**, on autorise la saisie d'une liste d'adresses de courriel séparées par une virgule. Cet attribut fonctionne aussi avec les autres types de champs de saisie.

- Saisir le code précédent au sein du formulaire de **`index.html`** **(avant le bouton envoyer).**
- Tester le résultat. Quel doit être le format d'une adresse de courriel valide ?

  ……..

3.3 – Saisir un numéro de téléphone : `type="tel"`

```
<p>
	<label for="zonePhone">Téléphone : </label>
	<input type="tel" id="zonePhone" name="phone" />
</p>
```

Etant donné la diversité des formats de numéros de téléphone utilisés dans le monde, il n'y a pas de contrôle spécifique pour ce type de champ, mais il permet de faire apparaître le clavier numérique sur les terminaux mobiles.

- Saisir le code précédent dans la page **`index.html`** et tester le résultat.

3.4 – Imposer un format à la saisie

L'attribut **`pattern`**, utilisable avec tous les types de champs, permet de contraindre la saisie à respecter un certain format. La valeur de cet attribut est **une expression régulière** qui donne le modèle auquel doit répondre le texte saisi.
Par exemple, un numéro de téléphone en France est une suite de 10 chiffres. L'expression régulière pour spécifier un chiffre est **[0-9]**, ce qui signifie n'importe quel caractère compris entre 0 et 9. De même, **[a-z]** signifie une lettre minuscule et **[A-Z]** une lettre majuscule.
Un numéro de téléphone français doit respecter le modèle **`[0-9]{10}`**. `{10}` signifie dix fois le modèle précédent.
Pour le téléphone, on écrira donc :

```
<p>
	<label for="zonePhone">Téléphone : </label>
	<input type="tel" id="zonePhone" name="phone" pattern="[0-9]{10}" />
</p>
```

Les expressions régulières sont un moyen puissant pour spécifier un format. Les valeurs admises pour l'attribut pattern sont celles définies par le langage JavaScript.

- En vous aidant du site (section syntaxe motif pattern) : https://www.toutjavascript.com/savoir/savoir22.php3
- Ecrire le pattern permettant de saisir un numéro de téléphone du type : **+33-(0)-123-456-789**

  ……

- Tester le résultat

3.5 – Aide à la saisie

Imposer un format permet d'éviter les erreurs de saisie mais le problème est que l'internaute ne peut pas deviner ce que l'on attend de lui. La solution est d'ajouter un attribut **`title`** à la balise input.

```
<p>
	<label for="zonePhone">Téléphone : </label>
	<input type="tel" id="zonePhone" name="phone" pattern="[0-9]{10}"
                                title="Numéro de téléphone à 10 chiffres sans espace" />
</p>
```

Lorsque l'internaute désigne le champ de saisie avec le pointeur de la souris, un message d'aide s'affiche dont le contenu est la valeur de l'attribut `title`.

Ce texte apparaîtra aussi dans le message d'erreur éventuel. Cet attribut peut être utilisé dans tous les types de champs.

- Tester l'attribut **title** sur les différents champs vus précédemment et vérifier dans les différents navigateurs.
- Noter vos commentaires dans le tableau ci-dessous :

Chrome	Firefox	Internet Explorer

3.6 – Saisir un mot de passe : `type="password"`

```
<p>
	<label for="zonePass">Mot de passe : </label>
	<input type="password" id="zonePass" name="pass" />
</p>
```

Le type **`password`** se comporte comme un type `text`, mais il permet de masquer la saisie à l'écran.

- Tester dans les différents navigateurs et noter vos commentaires dans le tableau ci-dessous :

Chrome	Firefox	Internet Explorer

3.7 – Saisir une adresse Web : `type="URL"`

```
<p>
	<label for="zoneWeb">Site : </label>
	<input type="URL" id="zoneWeb" name="web" />
</p>
```

En pratique, les contrôles effectués sur ce type de champ ne sont pas encore très poussés. Une URL est considérée comme valide si elle commence par un nom de protocole suivi d'un deux-points et éventuellement complétée par une chaîne de caractères quelconque.

- Tester dans les différents navigateurs et noter vos commentaires dans le tableau ci-dessous :

Chrome	Firefox	Internet Explorer

3.8 – Saisir une date : `type="date"`

```
<p>
	<label for="naiss">Date de naissance : </label>
	<input type="date" id="naiss" name="datenaiss" />
</p>
```

Certains navigateurs proposent un mini calendrier pour faciliter la saisie.

- Tester dans les différents navigateurs et noter vos commentaires dans le tableau ci-dessous :

Chrome	Firefox	Internet Explorer

3.9 – Saisir un nombre : `type="number"`

Le type `number` permet de saisir un nombre. Dans certains navigateurs, le champ est accompagné de 2 petits boutons permettant d'incrémenter ou de décrémenter la valeur du champ.

```
<p>
	<label for="zoneQt">Quantité : </label>
	<input type="number" id="zoneQt" name="quant" min="0" max="10" step="2" value="6" />
</p>
```

- Tester dans les différents navigateurs et noter vos commentaires dans le tableau ci-dessous :

Chrome	Firefox	Internet Explorer

3.10 – Saisie par ascenseur : `type="range"`

Le type range permet de aussi de saisir un nombre en modifiant un curseur.

```
<p>
	<label for="zoneQ2">Quantité2 : </label>
	<input type="range" id="zoneQt" name="quant2" min="0" max="10" step="2" value="6" />
</p>
```

- Tester dans les différents navigateurs et noter vos commentaires dans le tableau ci-dessous :

Chrome	Firefox	Internet Explorer

3.11 – Sélectionner une couleur : `type="color"`

Ce type d'attribut permet dans certains navigateurs d'accéder aux fonctions avancées de choix de couleur sous Windows ou Mac OS.

```
<p>
	<label for="zoneCouleur">Couleur : </label>
	<input type="color" id="ZoneCouleur" name="couleur" />
</p>
```

- Tester dans les différents navigateurs et noter vos commentaires dans le tableau ci-dessous :

Chrome	Firefox	Internet Explorer

3.12 – Sélectionner un champ par défaut : `autofocus`

L'attribut autofocus de la balise <**`input`**> permet de préciser le champ par lequel la saisie doit commencer au chargement du formulaire.

- Faire commencer la saisie sur le premier champ du formulaire et tester dans les différents navigateurs. Noter vos commentaires dans le tableau ci-dessous :

Chrome	Firefox	Internet Explorer

3.13 – Taille des champs

Pour chacun des champs que nous venons d'étudier, il est possible de définir :

- La taille du champ : Attribut **`size`**
- La longueur maximale de saisie : Attribut **`maxlength`**
- Une valeur par défaut : Attribut **`value`**

```
<p>
   <label for="zonePseudo">Votre pseudo : </label>
   <input type="text" id="zonePseudo" name="pseudo" size="10" maxlength="6" value="toto" />
</p>
```

➢ Tester dans les différents navigateurs. Noter vos commentaires dans le tableau ci-dessous :

Chrome	Firefox	Internet Explorer

3.14 – Effacer le formulaire : `type="reset"`

Il est possible d'ajouter dans le paragraphe contenant le bouton « **`submit`** », un bouton de type **`reset`** qui effacera toutes les valeurs saisies.

```
<p>
       <input type="submit" name="envoi" id="envoi" value="Envoyer"/>
       <input type="reset" name="efface" id="efface" value="Effacer"/>
</p>
```

➢ Tester dans les différents navigateurs. Noter vos commentaires dans le tableau ci-dessous :

Chrome	Firefox	Internet Explorer

4 – Zones de texte multilignes : balise <textarea>

Saisie d'un message sur plusieurs lignes : balise **`textarea`** plutôt qu'une balise **`input`**.

```
<p>
       <label for="msg">Votre message : </label>
       <textarea id="msg" name="message" cols="25" rows="5">Valeur par défaut…
       </textarea>
</p>
```

➢ Tester dans les différents navigateurs. Noter vos commentaires dans le tableau ci-dessous :

Chrome	Firefox	Internet Explorer

5 – Eléments d'options

En plus des zones de saisies, il existe trois autres éléments d'options à utiliser dans un formulaire.

5.1 – Les cases à cocher : `type="checkbox"`

C'est un type particulier de l'attribut « **`type`** » dans la balise <**`input`**>. Les cases à cocher offrent à l'utilisateur la possibilité de faire un choix multiple.

```
<p>
   Quels moyens de transport utilisez-vous régulièrement ? <br />
    <input type="checkbox" id="voiture" name="voiture" />
    <label for="voiture">Voiture</label><br />
    <input type="checkbox" id="bus" name="bus" />
    <label for="bus">Bus</label><br />
    <input type="checkbox" id="velo" name="velo" checked/>
    <label for="velo">Vélo</label><br />
    <input type="checkbox" id="skate" name="skate" />
    <label for="skate">SkateBoard</label><br />
</p>
```

Remarque : L'attribut **`checked`** permet de cocher une case par défaut.

- Tester dans les différents navigateurs. Noter vos commentaires dans le tableau ci-dessous :

Chrome	Firefox	Internet Explorer

5.2 – Les boutons radio ou boutons d'options : `type="radio"`

Contrairement aux cases à cocher, les boutons radio permettent de faire un choix (**et un seul**) parmi une liste de possibilités. La balise est toujours un <**`input`**> avec cette fois la valeur radio pour l'attribut « **`type`** ».

Les boutons d'options fonctionnent par « groupe » et pour un groupe donné, ils doivent avoir le **même attribut nom**. C'est le contenu de **l'attribut `value`** qui sera envoyé au programme destinataire.

```
<p>
   Dans quelle tranche d'âge vous situez-vous ? <br />
    <input type="radio" id="moins15" name="age" value="moins15"/>
    <label for="moins15">Moins de 15 ans</label><br />
    <input type="radio" id="15-25" name="age" value="15-25" />
    <label for="15-25">15-25ans</label><br />
    <input type="radio" id="25-40" name="age" value="25-40" checked />
    <label for="25-40">25-40ans</label><br />
</p>
```

Remarque : l'attribut **`id`** doit être différent pour chaque élément, et donc pour une fois il ne peut pas être identique à l'attribut **`name`**.

- Tester dans les différents navigateurs. Noter vos commentaires dans le tableau ci-dessous :

Chrome	Firefox	Internet Explorer

- Créer une nouvelle zone d'options offrant un choix de continents (Europe, Afrique, Amérique, Asie, Antarctique, Océanie) et permettant de répondre à la question : Sur quel continent habitez-vous ?

5.3 – Les listes déroulantes : `balise <select>`

A l'intérieur du <**`select`**> </**`select`**>, on écrira autant de balises <**`option`**> que d'éléments que l'on souhaite afficher dans la liste. Le contenu de chaque élément option est le texte affiché dans la liste déroulante.

Pour associer un code à chaque valeur affichée, il faut utiliser l'attribut **`value`** pour chaque option.

```
<p>
    <label for="transport">Choisir un moyen de transport : </label><br />
    <select id="transport" name="transport">
        <option value="1">Voiture</option>
        <option value="2">Bus</option>
        <option value="3" selected >Métro</option>
        <option value="4">Vélo</option>
    </select>
</p>
```

Remarques :

- L'attribut **`selected`** permet de définir l'option qui doit être sélectionnée au chargement de la page ;
- Pour définir une liste à choix multiple, ajouter l'attribut **`multiple`** à la balise `<select>` ;
- La balise <**optgroup**> </**optgroup**> permet de créer des groupes d'options à l'intérieur de la liste. La syntaxe est :

```
<optgroup label="nom du groupe">
<option>…</option>
…
</optgroup>.
```

➢ Tester dans les différents navigateurs. Noter vos commentaires dans le tableau ci-dessous :

Chrome	Firefox	Internet Explorer

➢ Créer une nouvelle liste déroulante permettant de choisir un pays dans une liste de pays regroupés par continent : Europe (France, Espagne, Italie), Amérique (Canada, Etats-unis, Mexique), Asie(Chine, Japon).

6 – Ajouter un champ de recherche : type="search"

Quand un site devient complexe, il peut être pratique pour les internautes de pouvoir faire une recherche sur un mot clé.

6.1 – Utiliser Google

Pour effectuer une recherche sur Google, il faut indiquer au formulaire que l'attribut **`action`** sera l'adresse URL du moteur de recherche de Google et que la méthode utilisée sera de type « **`get`** ».
Il faut ensuite préciser le nom du champ de recherche attendu par Google, soit « **`q`** » pour query.

```
<form method="get" action="http://google.fr/search">
   <p>
       <input type="search" name="q" placeholder="Rechercher" />
       <input type="submit" value="Go" />
   </p>
</form>
```

➢ Tester dans les différents navigateurs. Noter vos commentaires dans le tableau ci-dessous :

Chrome	Firefox	Internet Explorer

6.2 – Recherche google limitée à notre site

Pour limiter la recherche à notre site, il faut ajouter un champ caché **`type="hidden"`** avec un attribut **`value`** contenant l'adresse de notre site au format « **`site:adresse`** ».

```
<form method="get" action="http://google.fr/search">
   <p>
       <input type="search" name="q" placeholder="Rechercher" />
       <input type="hidden" name="q" value="site:http://www.univ-poitiers.fr" />
       <input type="submit" value="Go" />
   </p>
</form>
```

Remarques :

- Notre site n'étant pas publié, nous utiliserons pour la démonstration le site de l'université ;
- Sur certains navigateurs, il est possible de faire une saisie vocale en ajoutant au champ de type « **search** » l'attribut **`x-webkit-speech="x-webkit-speech"`**.

➢ Tester dans les différents navigateurs. Noter vos commentaires dans le tableau ci-dessous :

Chrome	Firefox	Internet Explorer

7 – Donner du style au formulaire

7.1 - Organiser le formulaire en plusieurs zones : balises `<fieldset>` et `<legend>`

Les balises **`<fieldset>`** **`</fieldset>`** permettent de délimiter un groupe de champs par un cadre. A l'intérieur de ces balises `<fieldset>`, les balises **`<legend>`**
`</legend>` permettent de nommer ce groupe.

```
<fieldset>
       <legend>Moyens de transports utilisés</legend>
              <input type="checkbox" id="voiture" name="voiture" />
              <label for="voiture">Voiture</label><br />
              <input type="checkbox" id="bus" name="bus" />
              <label for="bus">Bus</label><br />
              <input type="checkbox" id="velo" name="velo" checked  />
              <label for="velo">Vélo</label><br />
              <input type="checkbox" id="skate" name="skate" />
              <label for="skate">SkateBoard</label><br />
</fieldset>
<fieldset>
       <legend>Tranche d'âge</legend>
              <input type="radio" id="moins15" name="age" value="moins15"/>
              <label for="moins15">Moins de 15 ans</label><br />
              <input type="radio" id="15-25" name="age" value="15-25" />
              <label for="15-25">15-25ans</label><br />
              <input type="radio" id="25-40" name="age" value="25-40" checked  />
              <label for="25-40">25-40ans</label><br />
</fieldset>
```

➢ Tester dans les différents navigateurs. Noter vos commentaires dans le tableau ci-dessous :

Chrome	Firefox	Internet Explorer

7.2 – Définir un ordre de tabulation : Attribut `tabindex`

Dans un formulaire, il est possible de se déplacer uniquement grâce à la touche de tabulation (`Tab`) située à gauche du clavier. Touche `Tab` pour passer au champ suivant et `Shift + Tab` pour passer au précédent. Ceci est très utile pour rendre un formulaire accessible aux personnes souffrant d'un handicap.

L'attribut **`tabindex=nombre`** peut être ajouté à toutes les balises déjà étudiées. En mettant la valeur la plus petite sur le `tabindex` du premier champ (`tabindex=10`), puis `tabindex=20` sur le second, etc…, on définit de manière fiable l'ordre de saisie sur les différents champs du formulaire. Le choix de l'espacement de 10 en 10 permet de rajouter plus tard un champ (`tabindex=15`) en cas d'oubli.

```
<p>
	<label for="pname">Prénom : </label>
	<input type="text" id="pname" name="pname" tabindex="10" />
	<label for="name">Nom : </label>
	<input type="text" id="name" name="name" tabindex="20" />
</p>
```

7.3 – Utiliser CSS

Il est possible de changer l'apparence des champs en fonction de la validité des informations saisies par l'internaute.

Pour améliorer notre formulaire, nous allons utiliser les effets suivants :

- Encadrer en rouge les saisies invalides, pendant la saisie ;
- Encadrer en vert les saisies valides, pendant la saisie ;
- Encadrer en bleu les champs obligatoires.

➢ Créer un nouveau fichier au sein du **`td2`** nommé « **`style.css`** ».

➢ Saisir le code CSS suivant :

```
/* CSS à appliquer sur le cadre (outline) d'une balise input en cours de saisie (focus) valide (valid) */
input:focus:valid {
	outline: 2px lime solid
}
/* CSS à appliquer sur le cadre (outline) d'une balise input en cours de saisie (focus) non valide (invalid) */
input:focus:invalid {
	outline: 2px red solid
}
/* CSS à appliquer sur le cadre (outline) d'une balise input dont la saisie est obligatoire (required) */
input:required {
	outline: 2px blue solid
}
/* CSS à appliquer sur le cadre (outline) d'une balise input dont la saisie n'est pas obligatoire (optional) */
input:optional {
	outline: 2px yellow solid
}
```

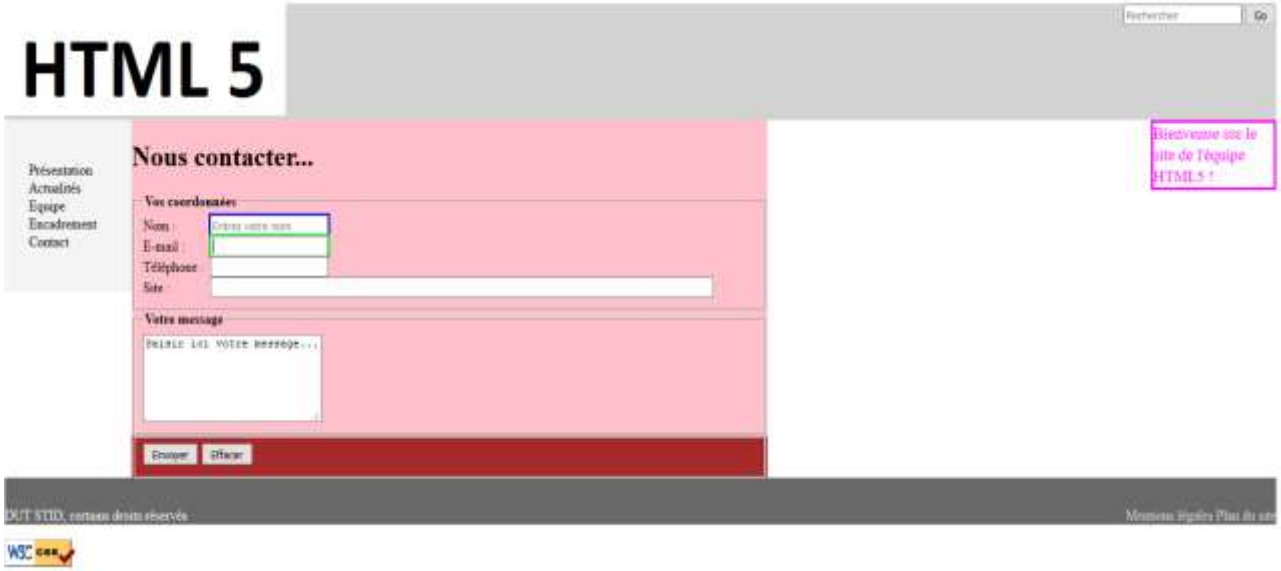
➢ Ajouter un lien dans la page **`index.html`** vers la feuille de style créée précédemment.

➢ Tester dans les différents navigateurs. Noter vos commentaires dans le tableau ci-dessous :

Chrome	Firefox	Internet Explorer

8 – Exercice

- Copier le dossier **`td1`** au sein du **`td2`** et le renommer **`siteWeb`**. Ajouter une page **`contact.html`** dans **`siteWeb`** pour afficher un formulaire permettant la prise de contact avec l'un des membres de l'équipe sur le modèle suivant :



Une fois validé le formulaire devra afficher sur une nouvelle page le texte « `Informations envoyées !` »